Deployment Tools: Linux, Docker & Kubernetes

Detailed study notes for DevOps, deployment, practical work, and viva preparation

Contents

1. Deployment and DevOps Fundamentals

Deployment means making an application available in an environment where users or other systems can access it. A deployment may involve preparing a server, installing dependencies, configuring networking, starting application processes, connecting a database, and monitoring the application.

DevOps combines development and operations practices to deliver software faster and more reliably. Linux is commonly used as the server operating system; Docker packages applications into portable containers; Kubernetes manages containers at scale.

Typical deployment flow

```
Developer → Git/GitHub → Build/Test → Docker Image → Container Registry → Kubernetes/Server → Application Users
```

Why these three technologies?

- Linux provides the operating-system environment and server administration tools.
- Docker provides consistent packaging and isolation for applications.
- Kubernetes automates deployment, scaling, service discovery, and recovery of containers.

Important distinction: Docker and Kubernetes are not replacements for Linux. Containers ultimately run using operating-system facilities, and Kubernetes commonly runs on Linux-based nodes.

2. Linux

2.1 What is Linux?

Linux is an open-source, Unix-like operating system kernel. In everyday usage, 'Linux' often refers to a complete Linux distribution such as Ubuntu, Debian, Fedora, or Red Hat Enterprise Linux. Linux is widely used for web servers, cloud infrastructure, containers, networking devices, and development environments.

2.2 Important Linux components

- Kernel: core of the operating system; manages CPU, memory, devices, processes and system calls.
- Shell: command-line interface such as Bash, used to execute commands and scripts.
- File system: organizes files and directories.
- Package manager: installs and updates software; examples include apt, dnf and yum.
- Services/daemons: background processes such as SSH, web servers and databases.

2.3 Essential commands

Command	Purpose	Example
pwd	Show current directory	pwd
ls	List files	ls -la
cd	Change directory	cd /var/www
mkdir	Create directory	mkdir app
cp	Copy files	cp app.js backup/
mv	Move/rename	mv old.txt new.txt
rm	Remove files/directories	rm file.txt
cat	Display file contents	cat package.json

Command	Purpose	Example
grep	Search text	grep error app.log
find	Find files	find . -name '*.log'
chmod	Change permissions	chmod +x deploy.sh
chown	Change owner	sudo chown user:user app
ps	Show processes	ps aux
top	Process/resource monitor	top
df	Disk usage	df -h
free	Memory usage	free -h
ip	Network configuration	ip addr
curl	Make HTTP requests	curl http://localhost:3000
ssh	Remote login	ssh user@server
systemctl	Manage services	sudo systemctl status nginx

2.4 Linux users and permissions

Linux uses users and groups to control access. Each file has an owner, a group, and permission bits for user (u), group (g), and others (o). The common permissions are read (r=4), write (w=2), and execute (x=1).

```
ls -l
chmod 755 deploy.sh
chmod 644 config.txt
sudo chown appuser:appgroup app/
```

755 means owner can read/write/execute, while group and others can read/execute. 644 means owner can read/write and group/others can read.

2.5 Processes and services

A process is a running program. Linux administrators inspect processes, stop misbehaving processes, and manage long-running services.

```
ps aux
top
sudo systemctl status nginx
sudo systemctl restart nginx
sudo systemctl enable nginx
```

2.6 Networking for deployment

- IP address identifies a network interface.
- Port identifies a network service, such as HTTP on 80 or HTTPS on 443.
- DNS maps names to IP addresses.
- SSH provides secure remote administration.
- Firewall rules control allowed network traffic.

```
ss -tulpn
curl http://localhost:3000
ssh user@server
```

2.7 Linux deployment example

```
sudo apt update
sudo apt install -y nginx
```

```
sudo systemctl enable --now nginx
sudo systemctl status nginx
```

For a web application, the application process can listen on an internal port while Nginx acts as a reverse proxy, forwarding public HTTP/HTTPS traffic to the application.

3. Docker

3.1 What is Docker?

Docker is a platform for building, packaging, distributing, and running applications in containers. A container packages an application with the libraries and configuration it needs, improving consistency between development, testing and deployment environments.

3.2 Container vs Virtual Machine

- Container: shares the host kernel and is generally lightweight and fast to start.
- Virtual machine: includes a guest operating system and virtualized hardware, so it generally requires more resources.

Containers provide process and filesystem isolation, but they are not full virtual machines.

3.3 Core Docker concepts

- Image: read-only packaged template used to create containers.
- Container: running instance of an image.
- Dockerfile: instructions for building an image.
- Registry: repository for images, such as Docker Hub or a private registry.
- Volume: persistent storage managed separately from a container's writable layer.
- Network: virtual networking that lets containers communicate.

3.4 Basic Docker workflow

```
docker pull node:22
docker build -t myapp:1.0 .
docker images
docker run -d --name myapp -p 3000:3000 myapp:1.0
docker ps
docker logs myapp
docker stop myapp
```

3.5 Dockerfile example for a Node.js application

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

FROM chooses the base image. WORKDIR sets the working directory. COPY transfers files. RUN executes build-time commands. EXPOSE documents the application port. CMD defines the default startup command.

3.6 Docker volumes

Container files can be lost when a container is removed. Volumes provide persistent storage for data that must survive container replacement.

```
docker volume create appdata
docker volume ls
docker run -d --name db -v appdata:/data myimage
```

3.7 Docker networks

```
docker network create appnet
docker run -d --name backend --network appnet mybackend
docker run -d --name db --network appnet mydb
```

Containers attached to the same user-defined network can communicate using container/service names, depending on the setup.

3.8 Docker Compose

Docker Compose defines and runs multi-container applications from a YAML configuration. It is useful for local development and simple multi-service environments.

```
services:
backend:
build: ./backend
ports:
- "3000:3000"
depends_on:
- mongodb
mongodb:
image: mongo:8
```

```
docker compose up -d
docker compose ps
docker compose logs
docker compose down
```

3.9 Docker advantages and limitations

- Advantages: portability, reproducibility, isolation, fast startup, efficient resource use and easy packaging.
- Limitations: containers share the host kernel; persistent data and security require careful configuration; running many containers manually becomes difficult, which is where orchestration tools help.

4. Kubernetes

4.1 What is Kubernetes?

Kubernetes, commonly abbreviated K8s, is an open-source container orchestration platform. It manages containerized workloads across a cluster and provides declarative configuration, scheduling, service discovery, scaling, rolling updates and self-healing.

4.2 Why Kubernetes?

- Automatically schedules workloads onto available nodes.
- Restarts failed containers according to desired state.
- Provides stable networking through Services.
- Supports horizontal scaling by changing the number of replicas.
- Supports rolling updates and rollbacks.
- Separates configuration and sensitive values from container images.

4.3 Kubernetes architecture

A Kubernetes cluster consists broadly of a control plane and worker nodes. The control plane manages cluster state and scheduling. Worker nodes run application workloads.

- API Server: central interface for Kubernetes operations.
- etcd: consistent key-value store containing cluster state.
- Scheduler: selects suitable nodes for new Pods.
- Controller Manager: runs controllers that reconcile actual state with desired state.
- Kubelet: node agent that ensures Pods/containers are running as specified.
- Container runtime: software responsible for running containers.
- Kube-proxy/networking components: support Service networking; exact implementation can vary.

4.4 Key Kubernetes objects

- Pod: smallest deployable unit; normally contains one main application container, though a Pod may contain multiple tightly coupled containers.
- Deployment: manages replicated Pods and supports rolling updates.
- Service: provides a stable network endpoint for a group of Pods.
- ConfigMap: stores non-sensitive configuration.
- Secret: stores sensitive configuration data; access and encryption practices still matter.
- Namespace: logical isolation/grouping within a cluster.
- Ingress: HTTP/HTTPS routing into cluster services, typically through an ingress controller.
- PersistentVolume/PersistentVolumeClaim: abstractions for persistent storage.

4.5 Basic Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: backend
spec:
replicas: 3
selector:
matchLabels:
app: backend
template:
metadata:
labels:
app: backend
spec:
containers:
- name: backend
image: myrepo/backend:1.0
ports:
- containerPort: 3000
```

4.6 Service YAML

```
apiVersion: v1
kind: Service
metadata:
name: backend-service
spec:
selector:
app: backend
ports:
- port: 80
targetPort: 3000
type: ClusterIP
```

4.7 Important kubectl commands

Command	Purpose
kubectl get pods	List Pods
kubectl get deployments	List Deployments
kubectl get services	List Services
kubectl describe pod NAME	Detailed Pod information
kubectl logs POD	View container logs
kubectl apply -f app.yaml	Create/update resources from YAML
kubectl delete -f app.yaml	Delete resources defined in YAML
kubectl scale deployment backend --replicas=5	Scale a Deployment
kubectl rollout status deployment/backend	Check rollout
kubectl rollout undo deployment/backend	Rollback a Deployment
kubectl exec -it POD -- sh	Open a shell in a container

4.8 Scaling and self-healing

If a Deployment specifies three replicas, Kubernetes attempts to maintain three matching Pods. If one fails, a controller can create a replacement. Horizontal scaling can increase or decrease replicas.

```
kubectl scale deployment backend --replicas=5
```

4.9 Rolling updates

A Deployment can gradually replace old Pods with Pods using a new image version. This reduces downtime and allows controlled rollouts.

```
kubectl set image deployment/backend backend=myrepo/backend:2.0
kubectl rollout status deployment/backend
kubectl rollout undo deployment/backend
```

4.10 Kubernetes vs Docker

Docker focuses on container creation and execution. Kubernetes focuses on orchestrating workloads across a cluster. Kubernetes can use container runtimes that implement the Kubernetes Container Runtime Interface; modern Kubernetes environments do not require Docker Engine specifically on every node.

5. Linux vs Docker vs Kubernetes

Technology	Main role	Typical level	Example
Linux	Operating system/server environment	Infrastructure	Run Nginx, Node.js, databases
Docker	Containerization	Application packaging/runtime	Build and run backend image
Kubernetes	Container orchestration	Cluster/platform	Run 3+ backend replicas and expose them through a Service

Easy analogy: Linux is the house/building foundation, Docker is a standardized box containing an application, and Kubernetes is the system that manages many such boxes across machines.

6. End-to-End Deployment Example

Scenario

Suppose a Student Skill Marketplace has a React frontend, a Node.js/Express backend, and MongoDB. A practical production architecture could place the frontend behind a web server/CDN, package the backend as a Docker image, and run backend replicas in Kubernetes. MongoDB may be hosted as a managed database rather than inside the same Kubernetes cluster.

Step 1: Prepare Linux server

```
sudo apt update
sudo apt upgrade
sudo apt install -y git
```

Step 2: Build Docker image

```
docker build -t student-marketplace-backend:1.0 ./backend
```

Step 3: Test container

```
docker run -d --name marketplace-api -p 3000:3000 student-marketplace-backend:1.0
curl http://localhost:3000
```

Step 4: Push image to registry

```
docker tag student-marketplace-backend:1.0 REGISTRY/marketplace-backend:1.0
docker push REGISTRY/marketplace-backend:1.0
```

Replace REGISTRY with the appropriate container registry/repository. Never place passwords, API keys, or database credentials directly in a Dockerfile or source repository.

Step 5: Deploy to Kubernetes

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get pods
kubectl get service
```

Step 6: Scale

```
kubectl scale deployment backend --replicas=3
```

Step 7: Monitor and troubleshoot

```
kubectl get pods
kubectl describe pod POD_NAME
kubectl logs POD_NAME
kubectl get events --sort-by=.lastTimestamp
```

A real production system also needs TLS/HTTPS, secrets management, backups, resource limits, health probes, monitoring, logging, access control, image scanning and a CI/CD process.

7. Important Viva Questions and Answers

What is Linux?

Linux is an open-source, Unix-like operating system kernel; Linux distributions provide the complete operating environment used on many servers.

Why is Linux popular for deployment?

It is stable, scriptable, widely supported by cloud platforms and has strong networking, process and permission controls.

What is Docker?

Docker is a platform for building, distributing and running applications in containers.

What is a Docker image?

An immutable packaged template used to create containers.

What is a Docker container?

A running, isolated instance of an image.

What is a Dockerfile?

A text file containing instructions used to build a Docker image.

What is Docker Compose?

A tool for defining and running multi-container applications using a Compose configuration.

What is Kubernetes?

An orchestration platform for managing containerized workloads across a cluster.

What is a Pod?

The smallest deployable unit in Kubernetes; it contains one or more containers that share networking and storage context.

What is a Deployment?

A Kubernetes controller/object used to manage replicated Pods and application rollouts.

What is a Service?

A stable network endpoint that exposes a set of Pods.

What is scaling?

Adjusting application capacity, such as changing the number of running replicas.

What is self-healing?

Kubernetes reconciliation mechanisms can replace failed Pods and maintain the desired state.

Docker vs Kubernetes?

Docker focuses on container packaging/runtime; Kubernetes orchestrates containerized workloads across a cluster.

Do you need Kubernetes for every application?

No. Small applications can often run directly on a server or with Docker Compose. Kubernetes becomes valuable when orchestration, scaling, resilience and multi-service management justify its complexity.

8. Quick Revision Sheet

- Linux = operating system/server foundation.
- Docker = package application into containers.
- Image = template; Container = running instance.
- Dockerfile = image build instructions.

• Registry = stores/distributes images.

• Kubernetes = orchestrates containers.

• Pod = smallest deployable unit.

• Deployment = manages Pods and rollouts.

• Service = stable network access to Pods.

• ConfigMap = non-sensitive configuration.

• Secret = sensitive configuration object; protect access carefully.

• kubectl = Kubernetes command-line tool.

• Replica = desired number of Pod instances.

• Rolling update = gradual replacement of old application version.

• Linux + Docker + Kubernetes form complementary layers, not interchangeable tools.